

NietoCity

Android port of Micropolis (the open-sourced SimCity engine)

Project plan and Phase 1 Claude Code prompt (Android and Windows 7)

Created by Nieto Software, 16 September 2026, revision 2 (adds Windows 7 desktop track, offline rule, Samsung S22 layout notes)

1. What this project is

An educational Android app that runs the genuine 1989 SimCity simulation. In 2008 Electronic Arts released the original source code under the GPLv3 as Micropolis, with the SimCity trademark and some artwork removed. Building on Micropolis gives you the real zone growth, traffic, pollution, power grid, budget and disaster logic rather than an imitation of it. This is the closest a legitimate project can get to the original.

The base is MicropolisJ, Jason Long's clean Java port. Its engine package has no user interface code and drops into an Android library module almost unchanged. The Swing interface is discarded and replaced with an Android renderer that recreates the classic look: tile map, top status bar (date, funds, population), tool palette on the left, mini map and disaster menu.

2. Licensing and naming

- Micropolis and MicropolisJ are GPLv3. If the APK is distributed (Play Store, F-Droid, or handed to students) the full source must be published under GPLv3. A public GitHub repository satisfies this.
- The SimCity name, logo, box art and Maxis title screen may not be used. The app ships as NietoCity with its own start screen. The Micropolis name may be used in credits.
- MicropolisJ's tile graphics derive from the open-sourced release and may be used. This is what makes the map look right.
- Every source file keeps its GPL header. THIRD_PARTY.md credits Electronic Arts, Maxis, Will Wright, Don Hopkins and Jason Long.

3. Visual fidelity targets

Element	Target
Map tiles	MicropolisJ tileset, 16 px tiles, integer zoom levels 1x, 2x, 3x so pixels stay crisp
Top bar	Month and year, funds, population, current tool name, in the classic order
Tool palette	Bulldozer, road, rail, power line, park, residential, commercial, industrial, police, fire, stadium, seaport, airport, coal and nuclear plant, query
Mini map	Overlays for zones, power, traffic, pollution, crime, land value, fire and police coverage
Disasters	Fire, flood, tornado, earthquake, monster, plane crash, nuclear meltdown, with original sprite animations
Dialogs	Budget, evaluation, graphs, city statistics, styled after the originals but drawn natively
Scenarios	Dullsville, San Francisco, Hamburg, Bern, Tokyo, Detroit, Boston, Rio de Janeiro (all present in Micropolis)
Sound	Open Micropolis sound set where available; otherwise short original clips created for Nieto Software

4. Technical stack

- Kotlin app module, pure Java :engine module, Gradle Kotlin DSL, minSdk 26, targetSdk 34.
- Rendering via SurfaceView and Canvas with a tile atlas bitmap; no game engine dependency needed.
- Simulation runs on its own thread at the original tick rate; UI observes state through a lightweight snapshot object.
- Touch: pan with one finger drag, pinch to change zoom step, tap to place the selected tool, long press for query.
- Saves in the original Micropolis .cty format so files can be exchanged with desktop Micropolis.
- Offline only: no INTERNET permission on Android, no update checks, no analytics; all resources bundled.
- Parallel Windows 7 build: a :desktop module (Java 8, JavaFX 8 or Swing) uses the same :engine module and tileset, packaged as a runnable JAR plus a launch4j .exe. One simulation, two renderers.
- Test phone: Samsung Galaxy S22 (1080 x 2340, about 425 dpi). Tiles render at 3x base zoom with 2x overview; in portrait the tool palette is a bottom sheet, in landscape the classic left column.
- Builds on the laptop or the work desktop (Android Studio, SDK platform 34, JBR JDK). A Gradle toolchain pins JDK 17 if the bundled JBR is too new for AGP.

5. Phase list (one Claude Code prompt per phase)

Phase	Goal	Done when
1	Scaffold, import engine, unit test, engine smoke screen	assembleDebug passes, test green
2	Tile renderer for Android (SurfaceView) and desktop (JavaFX Canvas) sharing one atlas; pan and zoom, animated tiles	A generated map renders and scrolls on phone and PC
3	Tool palette and placement, top status bar, tool cost and messages	A city can be built and grows
4	Speed control, budget, evaluation, graphs, mini map overlays	All classic dialogs function
5	Disasters menu, sprites (monster, tornado, plane, ship, helicopter), sounds	All disasters play correctly
6	New city generator, save and load (.cty), the eight scenarios	Scenarios load with their win conditions
7	Start screen, educational mode (explains each mechanic on tap), about and GPL screens	Ready for student testing
8	Polish, tablet layout, signed APK, Windows .exe via launch4j, source published	APK, exe and public repo

Phases 3 to 7 each deliver the feature on Android first and then on desktop within the same phase, so the two versions never drift apart.

Each phase prompt ends with a summary and a short list of things for you to test on the phone before the next phase is written.

6. Phase 1 Claude Code prompt

Paste this into Claude Code from an empty project folder.

```
# Phase 1: Project scaffold and Micropolis engine import

You are building "NietoCity", an Android and Windows 7 port of the GPL-licensed Micropolis
(the open-sourced original SimCity engine). Follow CLAUDE.md in this repo.
Warm, encouraging, concise tone. Never recursive deletes. Commit each task
with a clear message; I push via GitHub Desktop myself.

## Environment
- Windows, Android Studio installed. SDK at
  %LOCALAPPDATA%\Android\Sdk (platform 34, build-tools, platform-tools,
  cmdline-tools, emulator). ANDROID_HOME and JAVA_HOME (Android Studio JBR)
  are set as user environment variables.
- Check "java -version" first. If the JBR is newer than what the Gradle
  and AGP versions you pick support, configure a Gradle toolchain that
  downloads JDK 17 instead of changing my system variables.

## Pinned task list (do these in order, tick each off in TASKS.md)
  Test device: Samsung Galaxy S22, offline use only.
1. Create an Android project in the current folder: package za.co.nieto.nietocity,
  minSdk 26, targetSdk 34, Kotlin for the app module, Java allowed for the
  engine module. Gradle Kotlin DSL. The folder already contains CLAUDE.md,
  TASKS.md, README.md, LICENSE, THIRD_PARTY.md and .gitignore; keep them and
  update TASKS.md as you go. Git init if not already done.
  Offline only: do NOT add the INTERNET permission, analytics or update checks.
2. Add a pure-Java library module ":engine". Fetch the MicropolisJ source
  (Jason Long, github.com/jason17055/micropolis-java, GPLv3). Copy ONLY the
  micropolisj.engine package plus its resource files (tile spec files,
  disaster and terrain data) into :engine. Do not copy any Swing or GUI code.
  Keep the GPL headers on every file. Add LICENSE (GPLv3) and a THIRD_PARTY.md
  crediting Micropolis, Electronic Arts and Jason Long.
3. Make :engine compile with no android dependencies (java.awt must not be
  referenced; if any class uses it, replace with a small plain-Java shim and
  record the change in CHANGES.md).
4. Write a JUnit test in :engine that creates a new random map, places one
  residential zone, one road and one coal power plant, runs 200 sim ticks,
  and asserts that population and funds change. It must pass.
5. Create a minimal MainActivity that instantiates the engine, runs 100 ticks
  on a background thread, and shows "Engine OK: population X, funds Y" in a
  TextView. No rendering yet.
6. Add a pure-Java module ":desktop" targeting Java 8 (toolchain), depending
  on :engine, with a Main class that runs the same 100 ticks and prints the
  result to the console. Build it to a runnable JAR with "gradlew :desktop:jar".
  No desktop UI yet. This must run on Windows 7 with Java 8, so use no APIs
  newer than Java 8 in :engine or :desktop.
7. Confirm "gradlew assembleDebug" succeeds from the command line. Report the
  APK path. Do not open the file preview; just tell me it is written and where.
8. Update TASKS.md with the final status of every task and add a section
  "How to build" to README.md for both the APK and the desktop JAR.

## Rules for this phase
- Investigate and report the cause before changing anything when a build fails.
- Stop after task 8 and give me a short summary: what was done, what to test
  on the phone, and what Phase 2 needs from me.
```

7. Condensed CLAUDE.md

This file is already in the project folder; the content below is the reference version.

```
# CLAUDE.md (NietoCity)
- Tone: warm, encouraging, concise.
- Small phases, pinned task list in TASKS.md; tick tasks as they finish.
- Commit after each task; the developer pushes via GitHub Desktop.
- Never run recursive deletes (no rm -rf, no Remove-Item -Recurse).
- Bug fixes: investigate and report the cause before changing anything.
- After creating a file, say it is written and where; never open a preview.
- Every document produced also gets a PDF copy.
- All documentation carries "Created by Nieto Software".
- Licence: GPLv3 (Micropolis). Keep headers; credit EA, Maxis and Jason Long.
- No SimCity name, logo or Maxis artwork anywhere in the app or repo.
- Kotlin app module, Java engine module, Java desktop module (Java 8 APIs only).
- Android: minSdk 26, targetSdk 34. Desktop: must run on Windows 7 with Java 8.
- Offline only: no INTERNET permission, no analytics, no update checks.
- Test phone: Samsung Galaxy S22.
```

8. Things you can do now, away from the computer

1. Decide on the final app name if NietoCity is not it; it appears in the package name, so it is cheapest to change before Phase 1.
2. Create an empty public GitHub repository (GPL compliance) so the first push has a home.
3. Note which phone you will test on and its Android version; Phase 2 sizes the touch targets from that.